DHBW Ravensburg

Fortgeschrittene Informatik

Prof. Dr. Andreas Schilling

www.ravensburg.dhbw.de

2. Spring Data

Grundlagen	Spring Data (Repository)	Spring Data (Beziehungen)	Transaktionen
Grundlegende Prinzipien zu ORM und Spring-Data.	Vorgehen zum Speichern und Wiederherstellen von Objekten.	Vorgehen zum Speichern und Wiederherstellen von Objekt-beziehungen.	Wozu dienen Transaktionen und wie lassen diese sich in Spring nutzen.

Grundlagen

Ausgangszustand

Anwendung (objektorientiert)

```
1  public class Person{
2      private String surname;
3      private String forname;
4      private Collection<String> hobbies;
5  }
```

- Entitäten werden **objektorientiert** entworfen.
- Die **Fachlichkeit** bestimmt den Entwurf der Entitäten.
- Beziehungen zwischen Objekten werden mit **Referenzen** oder Vererbung abgebildet.

Datenbank (relational)

ID	Surname	Forename	Hobbies
1	Huber	Michaela	Lesen, Joggen
2	Müller	Franz	Fußball, Musik
...	...	...	...

- Tabellen werden in **relationalem Aufbau** strukturiert (SQL).
- Datenbankentwurf fokussiert sich auf **Normalformen** & **effiziente Speicherung**
- Beziehung zwischen Tabellen werden mit **Fremdschlüssel** abgebildet

Eine kritische Schnittstelle bei der Umsetzung von Web-Anwendungen betrifft die Speicherung von Daten. Dies liegt daran, dass sich die Anwendungsentwicklung, welche in der Regel objektorientiert umgesetzt wird, grundlegend vom Entwurf einer Datenbank, die im Gegensatz dazu oft einem relationalen Aufbau folgt, unterscheidet. Ein zentraler Unterschied ist, dass sich der objektorientierte Entwurf der Anwendung stark an der Fachlichkeit orientiert, während sich der Entwurf der Datenbanktabellen an Normalformen orientiert um so den vorhandenen Speicherplatz effizient zu nutzen. Abschließend unterscheiden sich Objekte zu Datenbanktabellen hinsichtlich der Umsetzung von Beziehungen. Während in der Anwendungsentwicklung Referenzen und Vererbungsbeziehungen zwischen Objekten bestehen, werden Beziehungen in Datenbanktabellen über Fremdschlüssel abgebildet.

Grundlagen

Objekt-Relationales Mapping (ORM)

ORM

Anwendung (objektorientiert)

```
1  public class Person{
2      private String surname;
3      private String forname;
4      private Collection<String> hobbies;
5  }
```

create →

← read

update →

delete →

Datenbank (relational)

ID	Surname	Forename	Hobbies
1	Huber	Michaela	Lesen, Joggen
2	Müller	Franz	Fußball, Musik
...	...	...	...

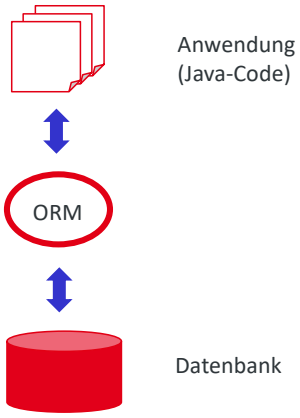
-> Der ORM übersetzt Datenstrukturen zwischen Anwendung und Datenbank

Die Lücke zwischen dem objektorientierten Anwendungsentwurf und dem relationalen Datenbankentwurf wird durch das Objekt-Relationale-Mapping (ORM) adressiert. Hierbei ist die zugrunde liegende Idee, dass ORM eine Zwischenschicht bildet um den Anwendungsentwurf mit den Datenbankentwurf zu kombinieren. Konkret übersetzt ORM Datenstrukturen zwischen der Anwendung und der Datenbank in beide Richtungen. Im Rahmen einer create Anweisung wird beispielsweise das Objekt aus der Anwendung in der Datenbank abgelegt. Hierbei werden Datentypen konvertiert, die von der Datenbank nicht unterstützt werden und Referenzen zu anderen Objekten werden in Fremdschlüsselbeziehungen aufgelöst. Bei der einer read-Anweisung werden die Datensätze aus der Datenbank in Anwendungsobjekte übersetzt. Wird ein update-Befehl ausgeführt wird der entsprechende Datensatz in der Datenbank identifiziert und die notwendigen Änderungsoperationen werden ausgeführt. Abschließend löscht die delete-Anweisung das übergebene Objekt aus der Datenbank.

Grundlagen

Objekt-Relationales Mapping (ORM)

- **Vorteile**
  - Harmonisierte Datenbankstruktur
  - Hoher Grad an Automatisierung
  - Komplexitätsreduktion der Datenschicht
  - Vermeidung von Angriffspotentialen
  - Erleichterte Austauschbarkeit der Datenbank
- **Herausforderungen**
  - SQL- Wissen ist unumgänglich
  - Datenbankabstraktion kann Performance schaden
  - Erst-Konfiguration kann aufwändig sein



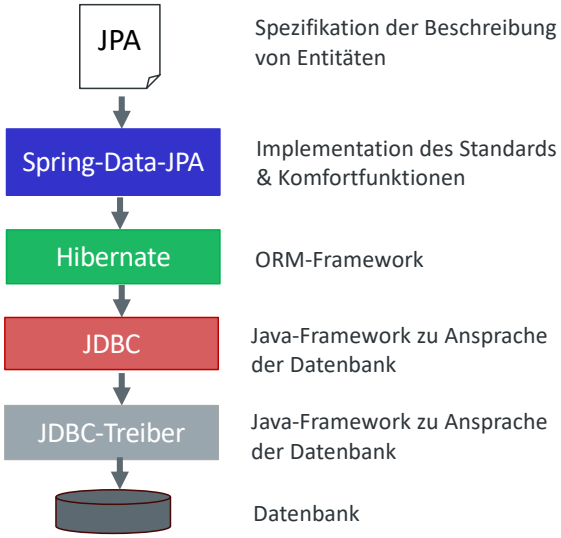
Die Einführung der ORM-Schicht zwischen Anwendung und Datenbank hat Vorteile aber auch Herausforderungen, welche wir uns im Folgenden genauer anschauen. Ein großer Vorteil von ORM ist die harmonisierte Datenbankstruktur, die aufgrund der durchgehenden Automatisierung erreicht werden kann. Im Vergleich hierzu erfordert es bei einem manuellen Mapping zwischen Objekten und Datenbanktabellen einen enormen Kraftakt eine konsistente Benennung, Transformation und Speicherung der Objekte umzusetzen. Darüber hinaus ist es durch den hohen Grad an Automatisierung möglich Änderungen an den Datenstrukturen mit vergleichsweise geringem Aufwand auch in späteren Stadien des Anwendungsentwurfs umzusetzen. Ein weiterer Vorteil, der durch ORM entsteht, ist, dass die Komplexität der Datenschicht, die der Entwickler verantwortet, massiv reduziert wird, da die Komplexität in den ORM verlagert wird. Mit der Übernahme der Verantwortung der Datenschicht durch den ORM wird zudem die Gefahr vor Angriffen (bspw. via SQL-Injection) reduziert. Abschließend wird durch ORM der Austausch der Datenbank erleichtert, da die Anwendung nicht mehr direkt sondern nur indirekt via ORM mit der Datenbank interagiert.

Neben den Vorteilen eines ORM gibt es aber auch Herausforderungen denen man sich bewusst sein muss. Eine wichtige Lessons-Learned ist, dass auch mit einem ORM, SQL-Wissen beim Anwendungsentwurf unumgänglich ist. Ferner entstehen durch den Komfortgewinn des ORM gewisse Performanceverluste gegenüber der nativen Ausführung. Abschließend kann festgehalten werden, dass für umfassende Anwendungen ein ORM unumgänglich ist, wobei jedoch die initiale Konfiguration des ORM aufwändig sein kann.

Grundlagen

Begriffe

- Die **Java Persistence API** (JPA) ist eine Spezifikation von Annotationen, die zur Persistierung verwendet werden.
- **Spring-Data-JPA** ist eine Implementierung des JPA-Standards und bietet weitere Komfortfunktionen.
- **Hibernate** ist ein populäres ORM-Framework zur Durchführung von ORM.
- **JDBC** ist ein Low-Level Framework zur Ansprache von Datenbanken.
- Der **JDBC-Treiber** verknüpft die Java-Laufzeit mit der Datenbank.
- Die **Datenbank** speichert die Daten in relationaler Form.



Schauen wir uns im Folgenden die Umsetzung von ORM im Kontext von Spring an und ordnen die eingesetzten Frameworks ein. Die Java Persistence API (JPA) ist eine Spezifikation, zur Beschreibung der Entitäten (d.h. der zu speichernden Objekte). Im Rahmen dieser Spezifikation werden formale Aspekte (bspw. Getter, Setter), als auch funktionale Aspekte bspw. Funktionsumfänge von Annotationen, beschrieben. Spring-Data ist eine Implementierung der JPA-Spezifikation, die darüber hinausgehende Komfortfunktionen bereitstellt. Die Umsetzung von JPA und das tatsächliche ORM erfolgt durch ein ORM-Framework bspw. Hibernate. JDBC ist ein Low-Level Framework zur Ansprache von Datenbanken (in dem bspw. nicht mit Entitäten gearbeitet wird, sondern mit ResultSets). Der JDBC-Treiber dient der Verbindung des JDBC-Frameworks mit der Datenbank. Abschließend werden die Daten in der Datenbank abgelegt.

Grundlagen	Spring Data (Repository)	Spring Data (Beziehungen)	Transaktionen
Grundlegende Prinzipien zu ORM und Spring-Data.	Vorgehen zum Speichern und Wiederherstellen von Objekten.	Vorgehen zum Speichern und Wiederherstellen von Objekt-beziehungen.	Wozu dienen Transaktionen und wie lassen diese sich in Spring nutzen.

Spring Data

Entities

- Entitäten sind jene Objekte, die in der Datenbank gespeichert werden. Sie tragen die Annotation **@Entity**
- Durch ORM wird für **jedes Klassenattribut eine eigene Spalte** mit dem korrespondierenden Typ in der Datenbanktabelle angelegt.
- Entitäten benötigen einen **leeren Konstruktor**, sowie **Getter** und **Setter** für die Attribute. Dies kann manuell oder via Annotation umgesetzt werden.
- Jede Entität besitzt einen **Identifikationsschlüssel (@Id),** der automatisch generiert werden kann (**@GeneratedValue**).

```java
@Getter
@Setter
@NoArgsConstructor
@Entity
public class Movie {

    @Id
    @GeneratedValue
    private long id;

    private String title;

    private double duration;

    private LocalDate premiereDate;

    private String description;

    private String trailerUrl;

    private String coverImgUrl;

    private double imdbRating;
```

Ausschnitt der Klasse Movie

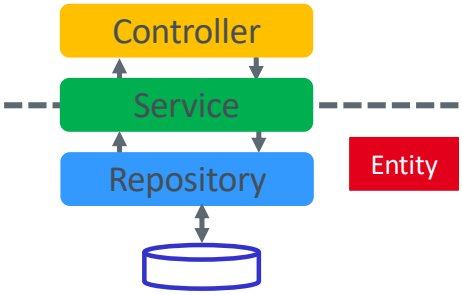
Entitäten bezeichnen jene Datenobjekte der Anwendung welche durch ORM in der Datenbank gespeichert werden, sie werden mit der Annotation @Entity gekennzeichnet und im Package „model“ abgelegt. Gemäß der JPA-Spezifikation müssen Entitäten ein besonderes Format aufweisen. Unter anderem benötigen alle Entitäten einen leeren Konstruktor, sowie Getter und Setter Methoden für die zu speichernden Klassenattribute. Diese Voraussetzungen können explizit umgesetzt werden oder implizit durch die Verwendung der Annotationen @NoArgsConstructor, @Getter und @Setter.

Eine weitere Vorgabe der JPA-Spezifikation ist, dass ein Klassenattribut als Identifikationsschlüssel (ein Merkmal, das die Entität in der Datenbank identifiziert) gewählt wird. Dies geschieht mit der Annotation @Id über dem entsprechenden Klassenattribut. Unter Nutzung der Annotation @GeneratedValue ist es möglich den Identifikationsschlüssel automatisch von Hibernate generieren zu lassen, um so Kollisionen mit anderen Objekten zu vermeiden.

Spring Data

Repositories

- **Repositories** abstrahieren die **Ansprache der Datenbank** zur Speicherung der Entitäten.
- Durch **No-Code Repositories** muss **nur das Interface spezifiziert** werden und Spring generiert automatisch die Implementierung.
- Der **Funktionsumfang des JpaRepositories** umfasst folgende Methoden:
  - save -> Speichert Entity
  - findById -> Stellt Entity aus Daten her
  - findAll -> Stellt alle Entities her
  - delete -> Löscht Entity




```
@Repository
public interface MovieRepository extends JpaRepository<Movie, Long> {
    List<Movie> findByTitleContaining(String substring);
    List<Movie> findByImdbRatingGreaterThan(double rating);
```

Ausschnitt des Interface MovieRepository

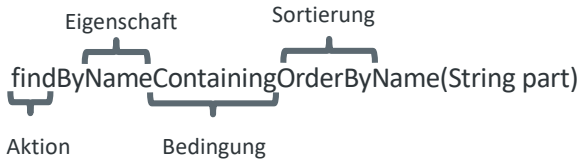
Ein wertvoller Mechanismus von Spring ist die Nutzung von Repositories zur Speicherung von Entitäten in einer Datenbank. Ein Repository beinhaltet alle Funktionen die zur Speicherung und zur Wiederherstellung von Entitäten aus einer Datenbank benötigt werden. Das Repository entkoppelt damit die Entität von der für die Speicherung erforderlichen Logik. Spring Data geht über die architektonische Trennung der Schichten hinaus und verwendet hierzu No-Code Repositories. Für diese No-Code Repositories erstellt Spring automatisch den notwendigen Implementierungscode. Der Entwickler muss hierfür nur ein Interface spezifizieren, welches das Interface JpaRepository erweitert. Dabei ist darauf zu achten, dass das Interface parametrisiert wird, wobei der erste Parameter der zu speichernde Entitätstyp und der zweite Parameter der Typ des Schlüsselmerkmals der Entität darstellt. Ohne weitere Konfiguration stehen nun die Funktionen zum Speichern (save), Wiederfinden via Schlüssel (findById), zur Ausgabe aller Entitäten (findAll) und zum Löschen von Entitäten (delete) bereit.

Spring Data

Abgeleitete Abfragen

Spring Data kann die **Implementierung** für Repository Methoden **automatisch ableiten**, wenn deren Namen dem folgenden Schema entspricht:

- **Aktion** beschreibt was durchgeführt werden soll (bspw. find, count, delete, exists).
- Die **Eigenschaft** bestimmt jenes Attribut der Entität auf das die Selektionsbedingung angewandt wird.
- **Bedingung** grenzt die Ergebnisse ein (bspw. Like, Containing, Between, GreaterThan).
- Die **Sortierung** bestimmt jenes Attribut nach dem die Ergebnisse sortiert bzw. gruppiert werden (bspw. OrderBy, OrderBy...Desc).



deleteByName(String name)

findByEmailNotNull()

findByAgeGreaterThan(int age)

findByAuthorStartingWith(String author)

findDistinctByNameAndAge(String name, int age)

Neben den Basisoperationen zum Wiederfinden von Objekten via Schlüssel und zur Rückgabe aller gespeicherten Objekte, kann das Interface um individuelle Methoden zur Rückgabe von Entitäten erweitert werden. Auch für diese Methoden kann Spring Data die notwendige Implementierung generieren. Voraussetzung hierfür ist, dass der Methodenname gemäß folgendem Schema aufgebaut ist:

- Die Aktion beschreibt was konkret mit der Entität gemacht werden soll. Mögliche Operationen sind: find, count, delete, exists
- Die Eigenschaft benennt das Attribut der Entity, das für den Vergleich ausgewertet wird.
- Die Bedingung beschreibt den zu nutzenden Vergleichsoperator bspw.: Containing, Between, GreaterThan
- Die Sortierung referenziert jenes Attribut anhand dessen die Ergebnisse gruppiert oder sortiert werden.

Die Dokumentation zu allen Bedingungen finden Sie unter unter https://docs.spring.io/spring-data/jpa/docs/current/reference/html/#jpa.query-methods

Spring Data

JPQL

- Durch die **@Query** Annotation können Abfragen manuell mit JPQL (SQL ähnliche Syntax) spezifiziert werden.
- JPQL operiert auf dem **Objektgraphen** und ist unabhängig von der konkret eingesetzten Datenbank.
- **Benannte Parameter** können mir der Annotation **@Param** verwendet werden
- Alle spezifizierten Abfragen werden beim Start der Spring Boot Applikation validiert.

```
@Query("SELECT m FROM Movie m WHERE m.title = :param1")
List<Movie> findAllMoviesByTitle(
        @Param("param1") String title);
```

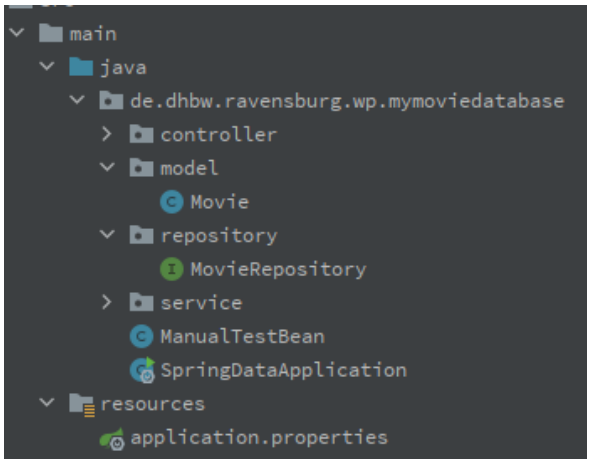
Ausschnitt des Interface MovieRepository

Die Nutzung von abgeleiteten Abfragen ist insbesondere bei simplen Abfragen sehr nützlich. Allerdings können die Methodennamen bei steigender Komplexität der Abfragen schnell unübersichtlich werden. In Situationen in denen komplexere Abfragen benötigt werden ist es zu empfehlen die Abfrage manuell mit der Annotation @Query zu definieren. Hierzu wird die Sprache JPQL verwendet, die große Ähnlichkeit zu SQL aufweist. Im Gegensatz zu SQL können bei JPQL die Abfragen basierend auf dem Objektgraphen erfolgen. Zudem können benannte Parameter in der Abfrage integriert werden. Die Richtigkeit aller spezifizierten Abfragen verifiziert Spring beim Start der Spring Boot Applikation und nicht erst bei deren Ausführung.

Spring Data

Hands-On

- Der Ordner ‚**model**' beinhaltet alle **Entitäten.**
- Der Ordner ‚**repository**' umfasst die **Repositories**, die zur Speicherung der Entitäten benötigt werden.
- Die Komponente „**ManualTestBean**" dient zur Erstellung eines Startzustands.
- Die Datei „**application.properties**" definiert die genutzte Datenbank (in-Memory h2) und deren Ansprache.



Verschaffen wir uns im Folgenden einen Überblick über die neue Ordnerstruktur in unserem Softwareprojekt. Der Ordner ‚model' umfasst alle Entitäten, die in der Datenbank gespeichert werden. In unserem Fall ist dies die Entität ‚Movie'. Der Ordner Repository gruppiert alle verwendeten Repositories (in der Regel nur Interfaces), die zur Speicherung der Entitäten genutzt werden. Zum Test der Repositories, sowie deren Methoden verwenden wir die ManualTestBean, da wir aktuell noch über keinen anderen Zugang zu unserer Applikation verfügen. Die Konfiguration der Datenbank und deren Ansprache wird in der Konfigurationsdatei application.properties im „resources" Ordner vorgenommen. In unserem Fall definieren wir dort die h2 Datenbank und dass beim Applikationsstart stets ein neues Datenbankschema erzeugt wird.

Spring Data

Hands-On

- Entitäten werden wie **normale Java Objekte (POJO)** instanziiert.
- Über die Methode **save** bzw. **saveAll** des entsprechenden Repositories werden die Entitäten gespeichert.
- Die Datenbank kann mit dem **Webbrowser** unter der Adresse http://localhost:8080/h2 eingesehen werden.
- Achten Sie darauf dass die **JDBC URL** in der Anmeldemaske wie folgt lautet: **jdbc:h2:mem:testdb**

```
Movie movie_6 = new Movie( title: "Star Wars: Rogue One",
        duration: 2.21,
        LocalDate.of( year: 2016, month: 12, dayOfMonth: 15),
        description: "The daughter of an Imperial scientist joins the Rebel
                "the plans for the Death Star.",
        trailerUrl: "https://www.imdb.com/video/imdb/vi830387737?playlistId=
        coverImgUrl: "https://upload.wikimedia.org/wikipedia/en/d/d4/Rogue_O
        imdbRating: 7.8);

//Filme speichern
this.movieRepository.saveAll(Arrays.asList(movie_1,movie_2,movie_3,movie_4,
```

Ausschnitt der Klasse ManualTestBean

Die Entitäten werden als gewöhnliche Java Objekte (Plain Old Java Objekt) umgesetzt. Das heißt, dass Entitäten bezüglich der Erstellung nichts zu normalen Java Objekten unterscheidet. Für die Speicherung der Objekte wird die save-Methode des korrespondierenden Repositories aufgerufen wodurch die Übertragung in der Datenbank erfolgt. Solange die Spring Boot Anwendung ausgeführt wird können Sie die Datenbank mit Ihrem Webbrowser unter der Adresse http://localhost:8080/h2 inspizieren.

Spring Data

Hands-On

Ausführung der Anwendung:

- Hibernate **löscht vorhandene Daten** (optional) und **erstellt** die für die Entitäten benötigten **Tabellen.**
- Zur Generierung neuer **Primärschlüssel** wird die **sequence** Tabelle aufgerufen.
- Die erstellten Entitäten werden in die Datenbank geschrieben.
- Die abgeleitete **Abfrage wird durchgeführt.**

Ausgabe Konsole

```
Hibernate: drop table director if exists
Hibernate: drop table movie if exists
Hibernate: drop sequence if exists hibernate_sequence
Hibernate: create sequence hibernate_sequence start with 1 increment by 1
Hibernate: create table movie (id bigint not null, cover_img_url varchar(255),
```

```
Hibernate: call next value for hibernate_sequence
Hibernate: call next value for hibernate_sequence
Hibernate: call next value for hibernate_sequence
Hibernate: call next value for hibernate_sequence
Hibernate: call next value for hibernate_sequence
Hibernate: call next value for hibernate_sequence
Hibernate: insert into movie (cover_img_url, description, duration,
Hibernate: insert into movie (cover_img_url, description, duration,
Hibernate: insert into movie (cover_img_url, description, duration,
Hibernate: insert into movie (cover_img_url, description, duration,
Hibernate: insert into movie (cover_img_url, description, duration,
Hibernate: insert into movie (cover_img_url, description, duration,
2020-12-16 14:51:40.112  INFO 10396 --- [  restartedMain] d.d.r.wp.m
Hibernate: select movie0_.id as id1_1_, movie0_.cover_img_url as cov
```


Schauen wir uns zum Abschluss die Ausgaben auf der Kommandozeile beim Start der Spring Boot Applikation an. Gemäß der Konfiguration in der application.properties Datei im Ordner „resources“ werden beim Start der Applikation bestehende Tabellen gelöscht. Dies ist nur für die Prototypentwicklung sinnvoll aber nicht für den produktiv Betrieb. Anschließend werden für die zu speichernden Entitäten die dazugehörigen Datenbanktabellen angelegt. Zudem wird für die Generierung der Primärschlüssel eine eigene Tabelle mit dem Namen sequence angelegt. Zuletzt finden Sie in der Konsole die Ausgabe zur Speicherung und Abfrage der Entitäten.

Aufgabe 2

Spring Data

a) Ändern Sie den Titel in Objekt **movie_5 nach Speicherung** des Objektes in „**Star Wars: Rise of the Skywalker**“ ab und **speichern Sie die Änderung**. Welche Schritte sind notwendig, damit diese Änderung in die Datenbank geschrieben wird?

b) Verfassen Sie eine **abgeleitete Abfrage** die jene Filme zurückgibt die **nach dem übergebenen Jahr** ihre Premiere hatten. Testen Sie die Methode in der **ManualTestBean** für das Jahr 2015.

c) Was für ein **Wertetyp** wird in der **Datenbank** für die Objektklasse LocalDate eingesetzt?

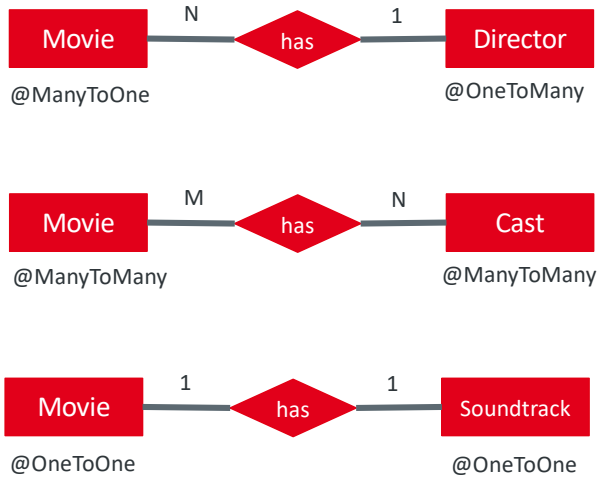
d) Erstellen Sie eine **JPQL Abfrage,** die jene Filme zurückgibt, welche kürzer als 2h und 30min sind und ein IMBD Rating größer 7 haben. Testen Sie die Methode in der **ManualTestBean.**

Grundlagen	Spring Data (Repository)	Spring Data (Beziehungen)	Transaktionen
Grundlegende Prinzipien zu ORM und Spring-Data.	Vorgehen zum Speichern und Wiederherstellen von Objekten.	Vorgehen zum Speichern und Wiederherstellen von Objekt-beziehungen.	Wozu dienen Transaktionen und wie lassen diese sich in Spring nutzen.

Spring Data

Beziehungstypen

- Entitäten stehen in der Regel miteinander in Beziehung.
- Beziehungen sind **uni-** oder **bidirektional.**
- Die **Kardinalität** gibt an wie viele Entitäten eines Typs mit Entitäten eines anderen Typs in Beziehung stehen:
  - **1:N** Eine Entität eines Typs steht mit vielen Entitäten des anderen Typs in Beziehung.
  - **M:N** Eine Entität eines Typs steht mit vielen Entitäten des anderen Typs in Beziehung und umgekehrt.
  - **1:1** Eine Entität eines Typs steht mit genau einer Entität des anderen Typs in Beziehung.



Nachdem wir uns im letzten Kapitel mit der Speicherung alleinstehender Objekte befasst haben, richten wir im Folgenden unseren Blick auf Objekte mit Beziehungen zueinander. Die Kardinalität gibt hierbei an wie viele Objekte des einen Typs mit wie vielen Objekten des anderen Typs in Beziehung stehen. Die gängigste Kardinalität zwischen Objekten ist die 1:N Beziehung bei der ein Objekt eines Typs mit vielen Objekten des anderen Typs in Beziehung steht. In umgekehrter Richtung steht ein Objekt genau mit einem Objekt des anderen Typs in Beziehung. Ein Beispiel für diesen Beziehungstyp ist die Beziehung Regisseur-Film. Ein Regisseur kann mehrere Filme drehen, wobei ein konkreter Film stets von genau einem Regisseur gedreht wird. Die M:N Beziehung ist sehr ähnlich zur 1:N Beziehung nur dass hier auch in der umgekehrten Richtung ein Objekt mit vielen Objekten des anderen Typs in Beziehung steht. Ein Beispiel hierfür ist die Beziehung zwischen Film und Schauspieler. Ein Film kann mehrere Schauspieler beinhalten und ein Schauspieler kann in mehreren Filmen mitwirken. Abschließend gibt es die 1:1 Beziehung bei der ein Objekt des einen Typs genau ein Objekt des anderen Typs referenziert und umgekehrt. Ein Beispiel hierfür ist, dass es für einen Film genau einen Soundtrack gibt und ein Soundtrack sich auch auf genau einen Film bezieht.

Spring Data

Bidirektionale Verbindungen

- Bei bidirektionalen Beziehungen muss stets ein Eigner bestimmt werden. Hierbei trägt die **abhängige Klasse** den Parameter „**mappedBy**“.
- Die **Objekt-Referenz** **<u>muss immer</u> auf der Eignerseite** gesetzt werden.
- Wenn der **Eigner nicht gespeichert wird** muss **zusätzlich** die Objekt-Referenz auf der **Kindseite gesetzt werden.**

Movie → Director

```
//Filme zuordnen
movie_1.setDirector(nolan);
this.movieRepository.save(movie_1);

movie_2.setDirector(nolan);
nolan.setMovies(Arrays.asList(movie_2));
this.directorRepository.save(nolan);
```

MovieRepository

Bei der Speicherung des Eigners werden automatisch die von ihm ausgehenden Beziehungen bidirektional übersetzt.

DirectorRepository

Bei der Speicherung der abhängenden Seite, muss die Referenz beidseitig gesetzt werden.

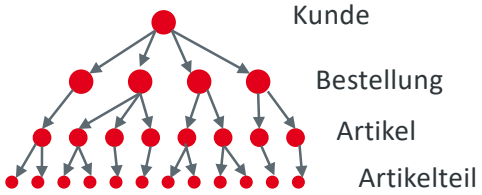
Für die Verwaltung von bidirektionalen Beziehungen ist es gemäß JPA-Spezifikation erforderlich eine Seite als Eigner der Beziehung zu bestimmen. Diese Definition ist insbesondere beim Durchreichen von Änderungen und Speicherbefehlen wichtig. Die abhängige Seite einer Beziehung wird mit dem Parameter „mappedBy“ bestimmt.

Beim Setzen der Referenzen im Programmfluss (siehe Grafik) muss die Referenz stets auf der Eignerseite zugewiesen werden. Das heißt in dem konkreten Beispiel Movie -> Director in der Movie der Eigner der Beziehung ist, muss stets die Director Referenz auf dem Movie Objekt via Setter gesetzt werden. Erfolgt die Speicherung über das MovieRepository ist dies ausreichend, da Movie der Eigner der Beziehung ist. Wird jedoch nicht die Movie Entität gespeichert sondern die Director Entität, via dem DirectorRepository, muss **zusätzlich** die Referenz auf der Seite des Director gesetzt werden.

Spring Data

Lazy vs. Eager Fetching

- Die **Ladestrategie** (FetchType) beschreibt, ob die referenzierten Objekte beim Laden einer Entität mitgeladen werden.
- Beim **Eager-Loading** werden die referenzierten Objekte der Beziehung mitgeladen.
- Im Falle des **Lazy-Loading** werden Platzhalter verwendet und die referenzierten Objekte werden erst beim tatsächlichen Zugriff geladen.
- In der Praxis führt die **Eager-Loading Strategie** bei **bidirektionalen** Beziehungen oft zu **hohen Performancedefiziten**.
- Generell sollte **Lazy-Loading** genutzt werden.



Die Eager-Loading Strategie kann im Praxis-einsatz schnell zu Performancedefiziten führen

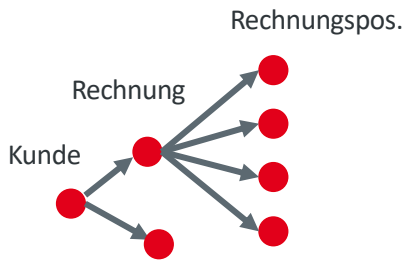
Die Ladestrategie beschreibt ob und welche referenzierten Objekte bei der Wiederherstellung einer Entität aus der Datenbank mitgeladen werden. Hierbei gibt es zwei grundlegende Ladestrategien. Beim Eager-Loading werden alle referenzierten Objekte beim Laden des Elternteils aus der Datenbank mitgeladen. Eager-Loading kann mit einer „einmal-alles Strategie“ verglichen werden, bei der beim ersten Zugriff alle referenzierten Objekte und deren Referenzen aus der Datenbank geladen werden. Diese Ladestrategie kann bei umfangreicher Vernetzung der Objekte (siehe Grafik) zu sehr umfangreichen Rückgabeobjekten führen. Bei der Lazy-Loading Strategie wird dagegen eine Referenz nicht im Vornhinein geladen sondern erst wenn tatsächlich ein Zugriff auf das entsprechende Attribut erfolgt. Das heißt, das Elternobjekt wird nur mit Platzhaltern geladen, die erst aufgelöst werden, wenn ein Zugriff auf das konkrete Attribut erfolgt. Generell empfiehlt es sich Objekte mit umfangreichen Beziehungen untereinander via Lazy-Loading zu laden.

Spring Data

Cascading

- **Cascading** (Durchreichung) beschreibt in wie weit sich Änderungen auf einer Seite der Beziehung auf die Entitäten der anderen Seite auswirken.
- Im Falle von **PERSIST** wird bei der Speicherung einer Seite stets auch die **referenzierten Objekte gespeichert** bzw. angelegt.
- Bei Nutzung von **REMOVE** wird beim Löschen einer Entität auch deren referenzierten Entitäten gelöscht.

  Achtung: Bei bidirektionalen Beziehungen kann das gravierende Folgen haben.



Die Durchreichungsstrategie muss gut überlegt sein, da sonst versehentlich viele Daten gelöscht werden.

Die Durchreichungsstrategie (eng. cascading) definiert ob bei Änderungen und der Speicherung auch die referenzierten Objekte berücksichtigt werden. Bei entsprechender Konfiguration ist es beispielsweise nur notwendig den Elternteil einer Beziehung zu speichern und dadurch auch gleich alle referenzierten Kinderknoten abzuspeichern. Dieser Komfortgewinn bei der Durchreichung kann aber auch zu negativen Folgen führen, beispielsweise beim Löschen von Objekten. Nehmen wir beispielsweise die Beziehung Kunde->Rechnung->Rechnungsposition. Bei entsprechender Durchreichung kann durch das Löschen eines Kunden automatisch sämtliche Rechnungen des Kunden und aller Rechnungspositionen mitgelöscht werden. Für die Buchhaltung hätte dies gravierende Konsequenzen da so wichtige Informationen zu Rechnungen verloren gehen. Im Falle einer bidirektionalen Beziehung ist ferner darauf zu achten, dass beim Löschen einer Rechnung nicht auch der Kunde gelöscht wird, da sonst ein gravierender Kreislauf entsteht.

Spring Data

Die 1:N Beziehung

- In der Entität wird diese Beziehung durch eine **getypte Liste** umgesetzt.
- In der Datenbank wir die Beziehung mit einer **Fremdschlüsselbeziehung** abgebildet.
- Die **Eignerseite** muss die Annotation **@JoinColumn** definieren.
- In der **Standardkonfiguration** gilt für die **ManyToOne** Seite **Eager-Loading** und für die **OneToMany** Seite ein **Lazy-Loading**.

Movie

ID	title	director_id
1	Interstellar	1
2	Dark Knight	1
...	...	...

Director

ID	surname	...
1	Nolan	...
2	Abrams	...
...	...	...

Movie

```
@ManyToOne(cascade = CascadeType.ALL)
@JoinColumn(name="director_id")
private Director director;
```

Director

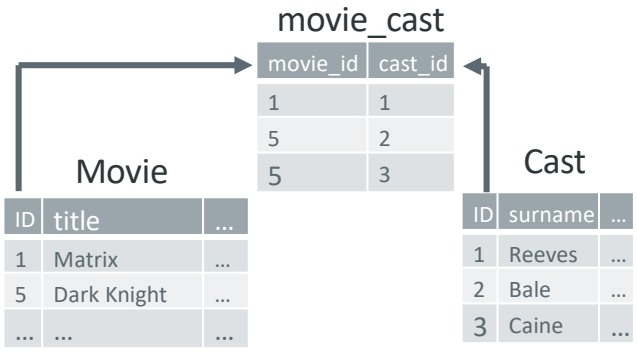
```
@OneToMany(mappedBy = "director", cascade = CascadeType.ALL)
private List<Movie> movies;
```


Die 1:N Beziehung oder auch OneToMany Beziehung ist die geläufigste Beziehung in der Anwendungsentwicklung. Hier steht ein Objekt eines Typs a mit vielen Objekten des Typs b in Beziehung, im Gegenzug steht jedoch ein Objekt des Typs b nur mit einem Objekt des Typs a in Beziehung. Ein Beispiel für eine derartige Beziehung ist die Verbindung zwischen Regisseur und Film. Ein Regisseur kann mehrere Filme drehen, wobei ein Film stets von genau einem Regisseur gedreht wird. Auf der Datenbank wird diese Beziehung über eine Fremdschlüsselbeziehung abgebildet, bei der die Eignerseite den Schlüssel der abhängende Seite (Kindseite) trägt (Fremdschlüssel). Diese Fremdschlüsselspalte wird bei der Eignerseite über die Annotation @JoinColumn definiert. In der Standardkonfiguration von Spring wird die ManyToOne Seite mit der Eager-Loading Strategie aus der Datenbank geladen und die OneToMany Seite mithilfe der Lazy-Loading Strategie.

Spring Data

Die M:N Beziehung

- In **beiden Entitäten** wird die Beziehung **je** durch eine **getypte Liste** umgesetzt.
- In der **Datenbank** wird die Beziehung über eine **neue Tabelle** umgesetzt, welche die Schlüssel der Entitäten beinhaltet.
- Die Eignerseite definiert die neue Tabelle mit der Annotation **@JoinTable**
- In der Standardkonfiguration werden beide Seiten der M:N Beziehung via **Lazy-Loading** geladen.



Movie

```
@ManyToMany(fetch = FetchType.EAGER, cascade = CascadeType.ALL)
@JoinTable(name = "movie_cast",
        joinColumns = {
                @JoinColumn(name = "movie_id", referencedColumnNa
                        nullable = true, updatable = false)},
        inverseJoinColumns = {
                @JoinColumn(name = "cast_id", referencedColumnNam
                        nullable = true, updatable = false)})
private List<Cast> involvedCast;
```

Cast

```
@ManyToMany(mappedBy = "involvedCast", fetch = FetchType.EAGER,
        cascade = CascadeType.PERSIST)
private List<Movie> participatedMovies;
```


Bei der M:N Beziehung oder auch ManyToMany Beziehung steht ein Objekt eines Typs a mit vielen anderen Objekten des Typs b in Beziehung und ein Objekt des Typs b wiederum mit vielen Objekten des Typs a. Ein Beispiel für eine derartige Beziehung ist die Verbindung zwischen Film und Schauspielern. In einem Film treten viele Schauspieler auf und ein Schauspieler kann wiederum in vielen Filmen mitwirken. Auf der Datenbank wird diese Beziehung über eine Fremdschlüsseltabelle umgesetzt in der die Primärschlüssel der beteiligten Entitäten verwendet werden. Diese Fremdschlüsseltabelle wird bei der Eignerseite über die Annotation @JoinTable definiert. In der Standardkonfiguration von Spring-Data werden beide Seiten der ManyToMany Beziehung via Lazy-Loading geladen.

Spring Data

Die 1:1 Beziehung

- In der Entität wird die Beziehung durch eine **einfache Objektreferenz** umgesetzt.
- In der **Datenbank** wird die Beziehung durch eine **Fremdschlüsselbeziehung** umgesetzt.
- Die **Eignerseite definiert** die notwendige Spalte via **@JoinColumn.**
- In der Standardkonfiguration werden beide Seiten der 1:1 Beziehung via **Eager-Loading** geladen.

Soundtrack

ID	author	movie_id
1	Zimmer	1
2	Manson	2
...	...	...

Movie

ID	title	...
1	Interstellar	...
2	Matrix	...
...	...	...

Soundtrack

```
@OneToOne(cascade = CascadeType.ALL)
@JoinColumn(name = "movie_id")
private Movie movie;
```

Movie

```
@OneToOne(mappedBy = "movie", cascade = CascadeType.ALL)
private Soundtrack soundtrack;
```


Die 1:1 Beziehung oder auch OneToOne Beziehung wird neben fachlichen Einsatzzwecken auch dazu verwendet komplexe Datenobjekte in kleinere Klassen zu strukturieren. Bei diesem Beziehungstyp steht ein Objekt des Typs a mit genau einem Objekt des Typs b in Beziehung. Im Gegenzug steht ein Objekt des Typs b mit genau einem Objekt des Typs a in Beziehung. Ein Beispiel für diesen Beziehungstyp ist die Verbindung zwischen Film und Soundtrack. Ein Film hat einen Soundtrack und ein Soundtrack bezieht sich auf genau einen konkreten Film. Auf der Datenbank wird diese Beziehung in Form einer Fremdschlüsselbeziehung umgesetzt bei der die Eignerseite der Beziehung den Schlüssel des Kindes der Beziehung trägt (Fremdschlüssel). Die Spalte für diese Fremdschlüsselbeziehung wird bei der Eignerseite über die Annotation @JoinColumn definiert. In der Standardkonfiguration von Spring werden beide Seiten der OneToOne Beziehung mit der Eager-Loading Strategie geladen.

Spring Data

Abfragen von Beziehungswerten

- Bei Beziehungen leitet Spring automatisch die **JOIN** Bedingungen ab, sodass **keine weiteren Angaben der Schlüssel** notwendig sind.

```
@Query("SELECT m FROM Movie m JOIN m.involvedCast c " +
        "WHERE c.surname = :param1")
```

Ausschnitt des Interface MovieRepository

- Bei **eindeutigen Zuordnungen** zwischen Entitäten kann über den „ **.** “ Operator im Objektgraphen navigiert werden.

```
@Query("SELECT m FROM Movie m " +
        "WHERE m.soundtrack.author_surname = :param1")
```

Ausschnitt des Interface MovieRepository

- Neben **Vergleichsoperationen** (bspw. <, >, =, <>) besitzt JPQL verschiedene **Funktionen** (bspw. size, length, locate) welche für Vergleiche eingesetzt werden können.

```
@Query("SELECT m FROM Movie m " +
        "WHERE size(m.involvedCast) > :param1")
```

Ausschnitt des Interface MovieRepository

Die JPQL Syntax bietet verschiedene Hilfestellungen zur Formulierung von Abfragen an, welche Beziehungen umfassen. Eine wichtige Erleichterung ist, dass bei einer JOIN Operation keine weiteren Angaben zur Übereinstimmung von Primär- und Fremdschlüssel spezifiziert werden müssen. Durch das Weglassen dieser Bedingungen wird die Abfrage deutlich vereinfacht. Darüber hinaus kann, wenn eine Entität eindeutig einer anderen Entität zugeordnet ist, ein JOIN komplett weggelassen werden und ausschließlich über die „.“ Operation im Objektgraphen navigiert werden. Abschließend bietet JPQL verschiedene Funktionsoperationen an, welche bei Abfragen eingesetzt werden können. So kann die size-Operation eingesetzt werden um die Größe von Objektsammlungen zu überprüfen. Eine gute Übersicht über die in JPQL zur Verfügung stehenden Funktionen und Vergleichsmöglichkeiten bietet die Website (https://thorben-janssen.com/jpql/#Inner_Joins).

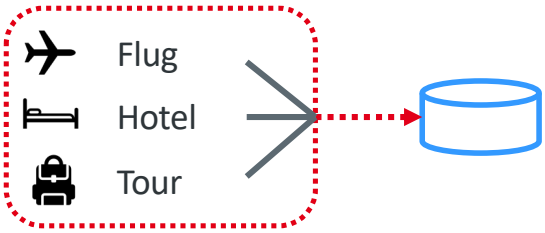
Grundlagen	Spring Data (Repository)	Spring Data (Beziehungen)	Transaktionen
Grundlegende Prinzipien zu ORM und Spring-Data.	Vorgehen zum Speichern und Wiederherstellen von Objekten.	Vorgehen zum Speichern und Wiederherstellen von Objekt-beziehungen.	Wozu dienen Transaktionen und wie lassen diese sich in Spring nutzen.

Transaktionen

Grundlagen

- **Transaktionen** ermöglichen es zusammengehörige Aktionen als eine **Einheit** umzusetzen.
- Der **Transaktionskontext** beschreibt die Hülle, welche die einzelnen Aktionen umschließt und als eine Einheit in die Datenbank schreibt.
- Zur Umsetzung einer Transaktion werden alle Aktionen über ein **Proxyobjekt** abgebildet.
- Werden nicht alle Aktionen der Transaktion erfolgreich durchgeführt, erfolgt ein **Rollback** auf den Startzustand.

Pauschalreise



Durch eine Transaktion können zusammengehörige Anweisungen als eine Einheit verarbeitet werden.

In bestimmten Situationen ist es wichtig, dass Datenbankaktionen nicht isoliert voneinander ausgeführt werden sondern als eine Einheit. Ein Beispiel hierfür ist die Buchung einer Pauschalreise, die Flug, Hotel und Sightseeing-Tour umfasst. In diesem Fall ist es nicht gewünscht, dass die Buchungen einzeln ausgeführt werden, da es sonst vorkommen kann dass ein Kunde einer Pauschalreise nur ein Flug und eine Tour zugewiesen bekommt aber kein Hotel. Transaktionen ermöglichen es einzelne Datenbankbefehle als eine Einheit auszuführen. Hierbei bezeichnet der Transaktionskontext die Hülle, welche die einzelnen Datenbankaktionen als eine Einheit zusammenfasst. Eine Transaktion wird in einem zweistufigen Vorgehen umgesetzt. Als Erstes werden alle Einzeloperationen im Transaktionskontext auf einem Stellvertreterobjekt (dem Proxy) ausgeführt. Ist dieser Vorgang erfolgreich wird im zweiten Schritt der Zustand des Proxyobjekts auf der Datenbank abgebildet. Die Änderungen werden dabei als eine atomare Einheit betrachtet, die nach dem Alles-Oder-Nichts Prinzip übertragen werden.

Können nicht alle Operationen der Transaktion erfolgreich in die Datenbank geschrieben werden, erfolgt ein Rollback auf den Startzustand der Transaktion.

Transaktionen

Grundlagen

Transaktionen erfüllen die ACID-Eigenschaften:

Atomar

Nur wenn **alle** Operationen der Transaktion erfolgreich sind werden diese gespeichert.

Tritt ein **Fehler** auf, werden die bisherigen Operationen der Transaktion **zurück geführt (Rollback)**.

Konsistenz

Jede Transaktion **beginnt und endet** auf einem **konsistenten** Datenbank-zustand.

Konsistent heißt, dass alle Einschränkungen und Bedingungen eingehalten werden.

Isolation

Jede Transaktion wird unabhängig von parallel ausgeführten Transaktionen durchgeführt.

Es gibt **keine Seiteneffekte** der Operationen auf parallel ablaufenden Transaktionen.

Dauerhaft

Die Ergebnisse der Transaktion sind **langfristig gesichert** und überdauern Serverausfälle.

Es wird folglich **kein Zwischenspeicher** und **kein flüchtiger Speicher** eingesetzt.

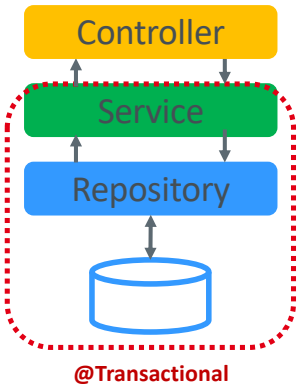
Transaktionen weisen vier zentrale Eigenschaften auf, die unter dem Namen ACID zusammengefasst werden.

- Atomar (Alles-oder-Nichts-Prinzip): Eine Transaktion wird als eine Einheit betrachtet. Das heißt nur wenn alle Aktionen der Transaktion erfolgreich sind wird diese umgesetzt.
- Konsistenz (Consistency): Eine Transaktion beginnt und endet stets auf einem konsistenten Datenbankzustand, dies gibt auch im Falle eines Rollbacks.
- Isolation (Unabhängigkeit): Transaktionen werden unabhängig voneinander ausgeführt, das heißt es gibt keine Seiteneffekte oder geteilten Variablen zwischen Transaktionen.
- Dauerhaft: Der Endzustand einer Transaktion ist dauerhaft. Es erfolgt bspw. keine Speicherung des Endzustands in flüchtigen Variablen.

Transaktionen

Umsetzung in Spring

- Bereits in der Standardkonfiguration erfolgen die **CRUD Befehle** an das Repository als Transaktion.
- Durch die Annotation **@Transacational** kann der Transaktionskontext auf eine **einzelne Methode** oder auf **alle Methoden** eines **Service** ausgeweitet werden.
- Der Transaktionskontext sollte **stets mit Bedacht** gewählt werden und **niemals als Standardfall**.
- **Lazy-Loading** Beziehungen können nur innerhalb des Transaktionskontexts aufgelöst werden.



Im Folgenden widmen wir uns der Umsetzung von Transaktionen in Spring. Wichtig hierbei ist, dass Transaktionen nur im Service und im Repository umgesetzt werden können, nicht in den Controllern. Bereits in der Standardkonfiguration werden sämtliche Create, Read, Update, und Delete Operationen der Repositories als Transaktionen umgesetzt. Dies hat zur Folge, dass neue Objekte nur komplett oder gar nicht in die Datenbank geschrieben werden. Es ist nicht möglich, dass nur Teile eines Objektes in die Datenbank geschrieben werden. Möchte man in einem Service mehrere Datenbankoperationen zu einer Transaktion bündeln wird die Annotation @Transactional über die entsprechende Methode im Service geschrieben. Alternativ kann man auch @Transactional über eine ganze Klasse schreiben was identisch zum Anbringen der @Transactional Annotation über allen Methoden der Klasse ist. Vorsicht jedoch vor der generellen Verwendung der @Transactional Annotation. Diese Annotation sollte stets mit Bedacht gewählt werden, da der Transaktionskontext auch zu unvorteilhaften Entwicklungen führen kann die wir uns auf der folgenden Folie genau anschauen. Abschließend muss berücksichtigt werden, dass Lazy-Loading Beziehungen von Entitäten nur innerhalb eines Transaktionskontexts aufgelöst werden können.

Transaktionen

Umsetzung in Spring

- **Alle Änderungen** an einer Entität werden beim erfolgreichen Abschluss einer Transaktion **gespeichert**. Es ist **kein expliziter Aufruf** von save notwendig
- Eine Transaktion kann **nur von außerhalb** einer Bean initiiert werden.
- **Niemals** @Transactional verwenden, **wenn nicht unbedingt** eine Transaktion benötigt wird.
- Durch die Annotation **@Transactional können Lazy-Loading** Beziehungen aufgelöst werden wie hier bspw. in der Methode getCastOfMovie.

```java
@Transactional
public void changePremiereDate(long movieId, LocalDate date){
    Movie tmp = this.movieRepository.findById(movieId).get();
    tmp.setPremiereDate(date);
    this.movieRepository.save(tmp);
}
```

Entweder @Transactional entfernen oder den save Aufruf

```java
//Im Transaktionskontext können LazyLoading Beziehungen aufgelöst werden
@Transactional
public List<String> getCastOfMovie(String movieTitle){
    List<String> result = new LinkedList<>();
    List<Movie> tmp = this.movieRepository.findByTitleContaining(movieTitle)
    if (tmp.size() == 1){
        tmp.get(0).getInvolvedCast().forEach(cast -> result.add(cast.getFore
    }
    return result;
}
```


Schauen wir uns zum Abschluss den praktischen Einsatz von Transaktionen in Spring an. Gemäß dem ACID-Prinzip werden alle Änderungen innerhalb einer Transaktion nach deren erfolgreichen Durchführung in der Datenbank gespeichert. Ein expliziter save Befehl ist nicht notwendig. Dies wird in der Praxis oft falsch gemacht, wie hier im oberen Codeauszug. In dem Beispiel ist der save Aufruf nicht notwendig solange die Annotation @Transactional verwendet wird. Das heißt es sollte entweder der save Aufruf oder die @Transactional Annotation entfernt werden.

Ferner ist es wichtig, dass Transaktionen nur von außerhalb einer Bean gestartet werden können, niemals durch Aufrufe innerhalb der gleichen Bean. Es ist wichtig, dass die @Transactional Annotation stets mit Bedacht an Methoden angebracht wird, da dies folgereiche Auswirkungen auf den Programmfluss hat.

Zuletzt schauen wir uns an wie wir im Transaktionskontext die Lazy-Loading Beziehungen auflösen können. Dies geschieht durch Anbringen der @Transactional Annotation über der entsprechenden Methode wodurch anschließend der Objektgraph, wie bei einem Eager-Loading, navigiert werden kann.

Aufgabe 3

Entity Beziehungen

a) Bitte fügen Sie in der Anwendung die neue Entität „**Award**“ hinzu welche in einer 1:N Beziehung zu Movie steht. Die neue Entität hat folgende Attribute:
   - academy (String)
   - category (String)
   - year (int)

b) Erstellen Sie in der ManualTestBean ein **neues Award-Objekte** und setzen Sie dieses mit movie_1 in Beziehung. Nehmen Sie die notwendigen Anpassungen vor, damit ihr neues Award Objekt gespeichert wird, sobald der referenzierte Film (movie_1) gespeichert wird.

c) Verfassen Sie eine JPQL-Abfrage welche alle Filme zurückgibt, die ein Award-Objekt mit dem Wert „Oskar“ beim Attribut academy zugewiesen haben.

d) Untersuchen Sie die Methode **changeMovieTitle** in der Klasse **MovieServiceImpl** auf Unstimmigkeiten oder Optimierungspotenziale.